Technisch Ontwerp

Van der Tweel Platform — Cemagi Atelier, Bali

Opdrachtgever	Max van der Tweel (ontwikkelaar, Cemagi Atelier)
Bedrijfsbegeleider	Bink
Datum	22 oktober 2026

Dit document bouwt voort op het M0-antwoorddocument (ons-antwoord.html) en vertaalt de zes functionele punten uit de bouwopdracht naar een technisch ontwerp: architectuur, datamodel, API's, beveiliging en niet-functionele eisen.

1. Overzicht frontend & backend

Het platform bestaat uit drie frontend-oppervlakken die dezelfde backend en database delen, zodat data nooit dubbel wordt bijgehouden.

1.1 Website (publiek)

Frontend: statische/server-rendered pagina's (project, prijzen en betaalplan, hoe kopen werkt, over Max, twee aanvraagformulieren). Geen login nodig.
Backend: twee schrijf-endpoints voor de aanvraagformulieren (bouwen met Max / grond-zoekopdracht), die een record aanmaken en een meldmail versturen.

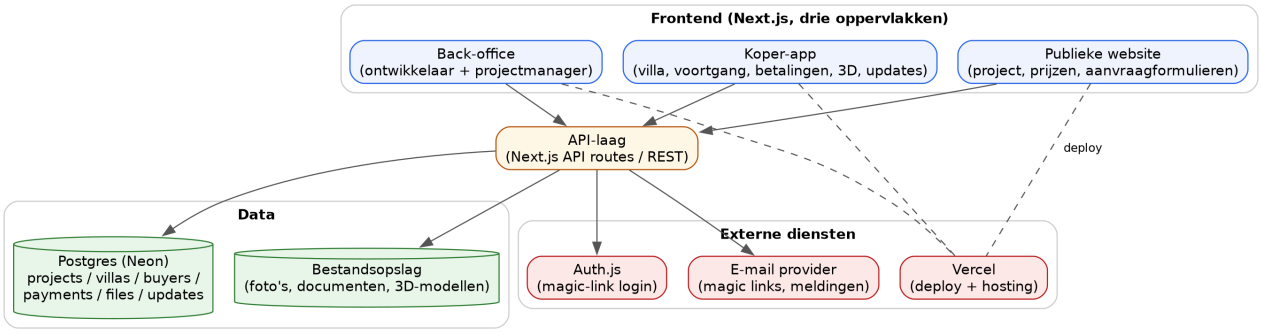
1.2 Koper-app

Frontend: acht schermen volgens Figma — eigen villa, voortgang, 3D-weergave, betaalschema, foto's, updates, documenten, account. Achter magic-link login.
Backend: alleen-lezen endpoints, strikt gefilterd op de villa(s) die aan het ingelogde koperaccount gekoppeld zijn (zie 9.1 voor de autorisatieregel).

1.3 Back-office (team)

Frontend: negen schermen voor de ontwikkelaar (volledig overzicht: projecten, villa's, kopers, betalingen, bestanden) en drie beperkte schermen voor de projectmanager (alleen zijn eigen projecten, geen geld, geen contractbestanden).
Backend: schrijf-endpoints voor voortgang, betalingen, updates en bestanden — elke schrijfactie triggert de koppeling naar de koper (zie hoofdstuk 4, dataflow).

1.4 Architectuuroverzicht



Alle drie de frontends praten uitsluitend met dezelfde API-laag; er is geen rechtstreekse databasetoegang vanuit de browser. Dit houdt de autorisatieregels op één plek afdwingbaar.

2. Overzicht database

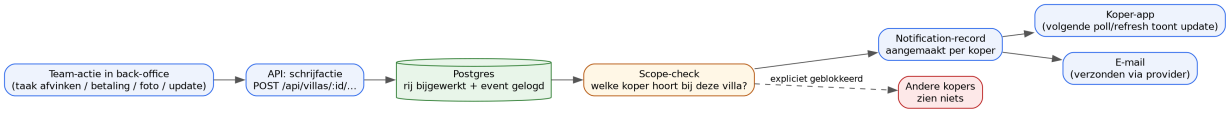
Eén Postgres-database (Neon) voor alle drie de oppervlakken. Kernentiteiten: Account, Project, Villa, Buyer (koper), ProjectManager, PaymentMilestone, ProgressTask, Update, File, Notification. Zie hoofdstuk 7 voor het volledige datamodel.

3. Overzicht externe software & tools

Dienst	Rol in het platform
Vercel	Hosting en deploy van website, koper-app en back-office.
Neon (Postgres)	Beheerde database, zelfde provider als andere lopende projecten.
Auth.js	Magic-link login zonder wachtwoord, rolgebaseerde sessie.
E-mailprovider (bijv. Resend)	Verzending van magic links en meldingen bij updates/betalingen.
Bestandsopslag (bijv. Vercel Blob / S3-compatibel)	Opslag van foto's, documenten en 3D-modellen.
React Three Fiber / Three.js	Interactieve 3D-weergave van de villa in de browser.

4. Logische verbinding & dataflow

De kern van het platform is niet een los scherm, maar de koppeling tussen een teamactie en wat de koper ziet. Dit is het klaar-criterium van milestone 5 (MS5) uit het plan van aanpak: een actie moet binnen een minuut bij precies de juiste koper staan, bij niemand anders.



De scope-check (wie hoort bij deze villa) gebeurt op API-niveau, niet pas in de interface — zo kan een verkeerd gebouwd schermpje nooit per ongeluk data van een andere koper tonen.

5. Deployment & hosting

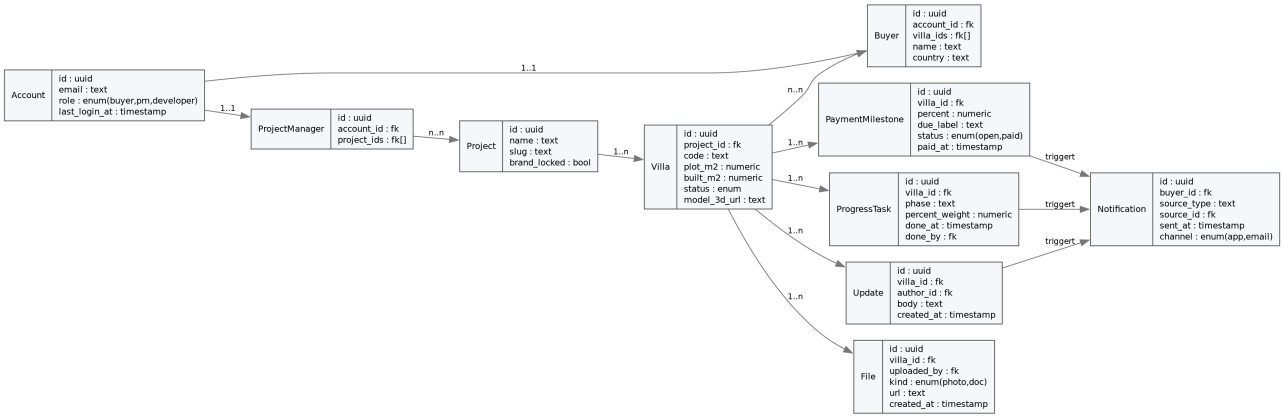
Website, koper-app en back-office worden als één Next.js-project gedeployed op Vercel, met gescheiden routes per oppervlak. Elke push naar de hoofdbranch triggert een preview-deploy; na review gaat dit naar productie. De database draait beheerd bij Neon, los van Vercel, zodat database-onderhoud geen deploy-downtime van de app veroorzaakt. Bestanden (foto's, 3D-modellen) worden niet in de database opgeslagen maar in losse bestandsopslag, met alleen de verwijzing (URL) in de database.

6. Techstack

Laag	Keuze	Reden
Taal / framework	TypeScript + Next.js (App Router)	Eén codebase voor website, app en back-office; goede mobiele performance; zelfde stack als eerdere projecten binnen het bureau.
Styling	Tailwind CSS	Snel component-ontwerp dat 1-op-1 op het Figma-ontwerp te leggen is.
3D	React Three Fiber (boven Three.js/WebGL)	Standaard voor interactieve 3D in React; geen ruwe WebGL omdat dat te low-level is voor dit project. LOD-fallback voor mobiel.
Database	Postgres (Neon)	Relationele data (koper hoort bij villa hoort bij project) past bij relationeel model; zelfde provider als andere projecten.
ORM	Drizzle of Prisma	Type-veilige database-toegang, migraties onder versiebeheer.
Auth	Auth.js, e-mail/magic-link provider	Sluit direct aan op de eis 'inloggen zonder wachtwoord'.
Hosting	Vercel	Zelfde patroon als bestaande demo's en andere projecten van het bureau.

7. Datamodel

7.1 Entiteiten (ERD, vereenvoudigd)



7.2 Tabellen en velden

Tabel	Belangrijkste velden
account	id, email, role (buyer/pm/developer), last_login_at
project	id, name, slug, brand_locked
villa	id, project_id (fk), code, plot_m2, built_m2, status, model_3d_url
buyer	id, account_id (fk), name, country — gekoppeld aan 1..n villa's via koppeltabel buyer_villa
project_manager	id, account_id (fk) — gekoppeld aan 1..n projecten via koppeltabel pm_project
payment_milestone	id, villa_id (fk), percent, due_label, status (open/paid), paid_at
progress_task	id, villa_id (fk), phase, percent_weight, done_at, done_by (fk account)
update	id, villa_id (fk), author_id (fk), body, created_at
file	id, villa_id (fk), uploaded_by (fk), kind (photo/doc), url, created_at
notification	id, buyer_id (fk), source_type, source_id, channel (app/email), sent_at

7.3 Indexering

Index op villa_id in payment_milestone, progress_task, update en file (meest voorkomende query: 'alles van deze villa'). Samengestelde index op buyer_villa (buyer_id, villa_id) voor de autorisatiecheck die bij elke koper-request draait. Index op account.email (unique) voor magic-link lookup.

7.4 Dataflow (samengevat)

Teamactie in back-office → schrijf-endpoint → database-update + event → scope-check welke koper(s) bij deze villa horen → notification-record per koper → zichtbaar in koper-app (volgende poll) + verzonden per e-mail. Zie het dataflow-diagram in hoofdstuk 4.

8. API's

8.1 Interne API (REST, intern gebruik door de drie frontends)

Endpoint	Methode	Rol	Doel
/api/auth/magic-link	POST	iedereen	Magic-link e-mail versturen.
/api/me	GET	ingelogd	Huidige account + rol + gekoppelde villa's/projecten.
/api/villas/:id	GET	koper (eigen), pm (eigen project), developer	Villadetails, voortgang, betaalschema.
/api/villas/:id/progress	POST	pm, developer	Voortgangstaak afvinken — triggert notification.
/api/villas/:id/payments/:mid	PATCH	developer	Betaalstatus wijzigen — triggert notification.
/api/villas/:id/updates	POST	pm, developer	Update/bericht plaatsen — triggert notification.
/api/villas/:id/files	POST	pm, developer	Foto/document uploaden — triggert notification.
/api/requests	POST	publiek (website)	Aanvraagformulier (bouwen met Max / grond zoeken).

8.2 Externe API's

Dienst	Gebruik
E-mailprovider (bijv. Resend)	Verzenden van magic-link mails en meldingen bij updates/betalingen/foto's.
Auth.js provider	Sessiebeheer en tokenvalidatie voor de magic-link flow.
Betaalprovider	Niet in scope voor v1 — betalen gebeurt buiten de app; open vraag naar Max welke methode (overboeking, Stripe) hij nu al gebruikt, voor eventuele koppeling in een latere fase.
CRM	Niet gekoppeld in v1; aanvraagformulieren komen nu per e-mail/record binnen, geen CRM genoemd in de brief.

9. Security

9.1 Authenticatie en autorisatie

Inloggen gebeurt uitsluitend via magic-link e-mail (Auth.js), geen wachtwoorden in het systeem. Na login bepaalt de rol op het account (buyer/pm/developer) welk oppervlak opent — er is geen rolkeuze op het inlogscherm. Autorisatie wordt op API-niveau afgedwongen: elke request op een villa controleert of het ingelogde account (koper of pm) daadwerkelijk aan die villa/dat project gekoppeld is, vóór er data wordt teruggestuurd. Een koper kan nooit de eigen scope omzeilen door een ander villa-ID in de URL te zetten (object-level authorization check, niet alleen UI-verberging).

9.2 Databeveiliging

Magic-link tokens zijn eenmalig en kortlevend (verlopen na gebruik of na een korte tijd). Geen wachtwoorden om te hashen, wat een hele risicocategorie wegneemt. Verbindingen uitsluitend via HTTPS/TLS. Persoonsgegevens van kopers (naam, land, contactgegevens, betaalstatus) vallen onder de AVG: dataminimalisatie (alleen opslaan wat nodig is voor het platform), een duidelijke bewaartermijn, en geen persoonsgegevens in logbestanden.

9.3 Input-validatie en rate limiting

Alle API-endpoints valideren input server-side (nooit alleen in de frontend vertrouwen), met typecontrole en lengtebeperking per veld. Rate limiting op /api/auth/magic-link en /api/requests om misbruik (spam, brute-force van e-mailadressen) tegen te gaan. Uploads (foto's/documenten) worden gecontroleerd op bestandstype en -grootte voordat ze worden opgeslagen.

10. Niet-functionele eisen (NFR)

10.1 Performance en snelheid

De koper-app moet op een gemiddeld mobiel toestel en een normale verbinding bruikbaar zijn: dit is een harde eis uit de brief, met name voor de 3D-weergave (lichtere modelversie als fallback bij trage verbindingen of oudere toestellen). Lijst- en detailschermen in de back-office laden met paginering in plaats van alles in één keer op te halen.

10.2 Logging en monitoring

Elke schrijfactie (voortgang, betaling, update, bestand) wordt gelogd met wie, wat en wanneer — dit dient zowel als audittrail (wie heeft wat gewijzigd) als als basis voor de notificatie-engine. Foutmeldingen in productie worden centraal verzameld (bijv. Sentry of vergelijkbaar), zodat een bug zichtbaar is vóór een koper of Max erover klaagt.

10.3 Backups en disaster recovery

Dagelijkse automatische backup van de Postgres-database (standaard bij Neon), met een bewaarperiode van minimaal enkele dagen terug, zodat een foutieve wijziging teruggedraaid kan worden. Geüploade bestanden (foto's, documenten, 3D-modellen) staan in aparte bestandsopslag met eigen redundantie, niet alleen in de database. Vóór livegang wordt één keer een herstel-oefening gedaan (backup daadwerkelijk terugzetten) om te bevestigen dat het werkt, niet alleen dat het geconfigureerd staat.

11. Testen & kwaliteitsborging bij AI-ondersteunde ontwikkeling

Een deel van de code wordt met AI-ondersteuning geschreven. Dat is geen reden om minder te testen — juist meer, met name op de onderdelen waar een fout een koper bij andermans data zou kunnen laten (autorisatie) of een betaling verkeerd zou laten tonen.

11.1 Unit tests

Elk API-endpoint dat data wijzigt (voortgang, betalingen, updates, bestanden) krijgt unit tests, met als vaste prioriteit de autorisatiecheck: hoort dit account echt bij deze villa? Dit wordt niet alleen handmatig getest, maar als geautomatiseerde test vastgelegd, zodat een latere wijziging dit nooit stilletjes kan breken.

11.2 Test-driven voor kritieke logica

Voor de onderdelen met het hoogste risico — de koppeling tussen teamactie en koperzicht (hoofdstuk 4), betaalstatus, en rolgebaseerde toegang — schrijven we eerst de test en dan pas de implementatie (TDD). Dit dwingt af dat 'hoort bij' en 'hoort niet bij' expliciet benoemd worden vóórdat er code is, in plaats van er achteraf aan te denken.

11.3 Eval tests voor AI-gegenereerde code

AI-gegenereerde code die productiedata raakt (met name autorisatie en betalingen) wordt niet blind aangenomen. Elke AI-bijdrage op deze onderdelen doorloopt: (1) de bestaande of nieuw geschreven unit tests moeten slagen, (2) een vast evaluatiescenario — 'kan koper A, via welke route dan ook, data van villa B bereiken?' — moet falen (d.w.z. nee opleveren), (3) menselijke review vóór merge naar main. Dit scenario is het belangrijkste terugkerende testgeval van het hele project en wordt bij elke wijziging aan autorisatie of API-routes opnieuw gedraaid.

11.4 CI en staging

Tests draaien automatisch bij elke pull request; een merge naar main wordt geblokkeerd zodra een test faalt (zie ook 4.6 in het Plan van Aanpak voor de bredere git-workflow). Er is een aparte staging-omgeving met eigen database, zodat nieuwe functionaliteit getest wordt zonder ooit Max' echte koperdata of betaalgegevens te raken. Pas na een groene staging-test gaat een wijziging naar productie.